

RAG 實作教學

五個範例的實作方式與預期結果

dilonwork/rag-example 實作導覽 · 2026-10-08

這份文件對應 repo 裡 examples/

的五個範例。每個範例都包含：要解決什麼問題、關鍵程式碼如何運作、執行指令，以及你應該會看到的預期輸出。讀完後你會知道 RAG pipeline 每個階段實際在做什麼，以及怎麼換掉其中任何一個模組。

一、架構總覽

RAG

分成兩個階段。索引階段（一次做完）：把文件變成可搜尋的向量；查詢階段（每次問問題都跑）：找出相關片段，交給 LLM 生成答案。

索引階段 (ingest)

data/docs — 1.ingestion —> 文件物件
— 2.chunking —> 重疊文字塊 (chunks)
— 3.embeddings —> 密集向量 (vectors)
— 4.vector store —> 入庫，可做餘弦相似度搜尋

查詢階段 (ask)

問題 — 5.retrieval —> 取出 top-k 相關 chunks
— 6.prompting —> 組成「背景+問題+引用要求」的提示
— 7.llm —> 生成最終答案

- 七個階段各是一個獨立模組 (src/rag_example/ 下七個 .py 檔)，介面都很小。
- RAGPipeline (pipeline.py) 負責把七個階段串起來；換掉任一模組不用動其他程式。
- 範例文件在 data/docs/：3 份英文文件，預設切分後產生 6 個 chunks（見下一節）。

二、前置準備

在 repo 根目錄執行以下指令安裝依賴（只需要做一次）：

```
pip install -r requirements.txt
```

範例文件經預設切分器（800 字元、重疊 120 字元）處理後，會產生以下 6 個 chunks：

```
chunking-strategies#0 793 字元 (Chunking Strategies 上半)
chunking-strategies#1 320 字元 (Chunking Strategies 下半)
vector-databases#0 799 字元 (Vector Databases 上半)
vector-databases#1 343 字元 (Vector Databases 下半)
what-is-rag#0 796 字元 (What is RAG 上半)
what-is-rag#1 325 字元 (What is RAG 下半)
```

註：下面所有範例都在 repo 根目錄執行。embedding 模型第一次使用會下載約 90MB，之後會快取。

三、範例一：01_retrieve.py —— 從 RAG 取出內容

3.1 這個範例要學什麼

理解 retrieval 到底回傳什麼。答案不是「一段文字」，而是「排名過的 chunk 清單＋相似度分數＋來源資訊」。這是整個 RAG 系統的輸出入關鍵，也是之後做引用、除錯的基礎。

3.2 實作解說

程式只有三個動作，對應 pipeline 的階段 1-5：

關鍵程式碼

```
pipeline = RAGPipeline(Settings()) # 組裝七個模組，參數全走環境變數
pipeline.ingest() # 階段 1-4：載入→切分→向量化→入庫
hits = pipeline.retrieve(QUESTION) # 階段 5：問題向量化→取 top-k
```

retrieve() 內部做的事：先用同一個 embedding 模型把問題轉成向量，再到 vector store 做餘弦相似度搜尋，回傳前 k 個最相關的 chunk。每個回傳項目是 ScoredChunk，包含 chunk（文字＋標題＋來源路徑＋編號）與 score（相似度分數）。

3.3 執行

```
python examples/01_retrieve.py
```

3.4 預期結果

預期輸出（分數為示意，實際數值依 embedding 模型而異）

```
indexed 6 chunks

Q: How should I choose a chunk size?

--- rank 1 | similarity 0.71 ---
from: Chunking Strategies (data/docs/chunking-strategies.md, chunk 0)
# Chunking Strategies
```

Chunking decides how documents are split before embedding, and it has an outsized effect on retrieval quality. ...

--- rank 2 | similarity 0.58 ---

from: Chunking Strategies (data/docs/chunking-strategies.md, chunk 1)
...chunks small enough that one chunk holds one idea (200-800 characters is typical for Q&A), and always store metadata ...

--- rank 3 | similarity 0.44 ---

from: What Is Rag (data/docs/what-is-rag.md, chunk 0)
What is RAG?
... Its main failure modes are bad chunking, weak retrieval ...

註：chunk 文字與 metadata 格式如實；排名與分數會因 embedding 模型與問題用詞略有差異，但談 chunk size 的兩個 chunk 排在最前面是預期中的。

3.5 觀念重點

- score 是餘弦相似度 (0-1) 。不用糾結絕對值，看「相對排名」就好。
- 每個 chunk 都帶著 title、source、index——這些 metadata 是下一節做「引用來源」的基礎。
- 如果排名不如預期，第一個要懷疑的是 chunking（大小／重疊／策略），不是 embedding 模型。

四、範例二：02_answer.py —— 完整問答

4.1 這個範例要學什麼

走完查詢階段的全部：取出內容 → 組成提示 → LLM 生成答案。你會親眼看到送進 LLM 的 prompt 長什麼樣子，以及答案如何標註引用來源。

4.2 實作解說

pipeline.ask() 內部就是三行，對應階段 5-7：

關鍵程式碼 (pipeline.py)

```
hits = self.retrieve(question)          # 階段 5：取出 top-k
system_prompt, user_prompt = build_rag_prompt(question, hits) # 階段 6：組提示
text = self.llm.generate(system_prompt, user_prompt) # 階段 7：生成答案
```

build_rag_prompt 做的事很單純：把每個 chunk 編號為

[1][2]...，連同標題一起拼成背景文字，再加上問題與指示。實際送出的提示長這樣：

實際 prompt 結構（以範例文件內容示意）

[系統提示]

You are a helpful assistant that answers questions using ONLY the context

below. Cite the sources you used with bracketed numbers like [1], [2].
If the context does not contain the answer, say so instead of guessing.

[使用者提示]

Context:

[1] (Chunking Strategies)

Chunking Strategies

Chunking decides how documents are split before embedding ...

[2] (What Is Rag)

What is RAG?

... Its main failure modes are bad chunking, weak retrieval ...

Question: What are the failure modes of RAG and how do I fix them?

Answer concisely and cite sources with bracketed numbers.

4.3 LLM 設定

- 預設走本機 Ollama：先跑 ollama serve，再 ollama pull llama3.1。
- 或切到 OpenAI 相容 API：RAG_LLM_PROVIDER=openai RAG_LLM_MODEL=gpt-4o-mini OPENAI_API_KEY=sk-...。
- Ollama 沒開會看到清楚的錯誤訊息，告訴你該啟動服務或改用 OpenAI。

4.4 執行與預期結果

python examples/02_answer.py

預期輸出（答案文字每次可能不同，引用格式與來源列表是重點）

indexed 6 chunks

Q: What are the failure modes of RAG and how do I fix them?

A: The main failure modes are bad chunking, weak retrieval, and prompts that let the model ignore the context [2]. To fix them, start by experimenting with chunking ... [1].

Sources used:

[1] Chunking Strategies — data/docs/chunking-strategies.md (score 0.66)

[2] What Is Rag — data/docs/what-is-rag.md (score 0.61)

...

4.5 觀念重點

- 系統提示裡的 ONLY 與「不知道就說不知道」是 RAG

的靈魂：強迫模型只用檢索到的內容回答，減少幻覺。

- [1][2] 引用編號對應傳入的 chunk 順序，由 prompting.py 負責編號，前後一致。

五、範例三：03_my_docs.py —— 換成自己的文件

5.1 這個範例要學什麼

把範例文件換成你自己的筆記或文件。RAG

的價值就在這裡：回答「模型沒學過、只有你有的」內容。

5.2 實作解說

關鍵只有一個環境變數。Settings 會讀 RAG_DOCS_DIR，ingestion 會遞迴讀取該目錄下所有 .md/.txt 檔：

關鍵程式碼

```
settings = Settings()      # RAG_DOCS_DIR 沒設就用 data/docs
settings.top_k = args.top_k # 可順手調整取回幾個 chunk
pipeline = RAGPipeline(settings)
pipeline.ingest()
```

5.3 執行

```
RAG_DOCS_DIR=~/.my-notes python examples/03_my_docs.py --ask "快取怎麼設計的？"
# 可選：--top-k 6 取回更多 chunks
```

5.4 預期結果

預期輸出

```
indexed 42 chunks from /home/user/my-notes
```

```
Q: 快取怎麼設計的？
```

```
[0.73] redis 筆記 (my-notes/cache.md)
```

```
快取採用兩層設計：L1 本機記憶體 ...
```

```
[0.61] 系統設計 (my-notes/design.md)
```

```
...
```

5.5 觀念重點

- top-k 是取捨：k 太小會漏關鍵資訊，k 太大會塞進雜訊、稀釋 prompt。4 是常見起點。
- 文件是中文也沒問題，但 embedding 模型最好選多語言的（預設的 MiniLM 以英文為主）。

六、範例四：04_custom_chunker.py —— 自訂切分策略

6.1 這個範例要學什麼

證明 pipeline 的模組真的可以插拔：不動 pipeline 本體，只寫一個新的切分器插進去。

6.2 實作解說

chunking.py 定義了 Chunker 協定：任何有 .chunk(documents) 方法的物件都能用。範例實作了 SentenceChunker——按句子分組，每 2 個句子一塊：

關鍵程式碼

```
class SentenceChunker:
    """One chunk per N sentences."""
    def chunk(self, documents: list[Document]) -> list[Chunk]:
        ... # 句子切分後每 2 句一組，包成 Chunk 物件

# 注入：pipeline 其他部分完全不用改
pipeline = RAGPipeline(Settings(), chunker=SentenceChunker(sentences_per_chunk=2))
```

RAGPipeline.__init__ 接受 chunker= 參數，沒給就用預設的 FixedSizeChunker。換 embedding 或 vector store 也是同樣手法：實作對應協定，構造時傳進去。

6.3 執行

```
python examples/04_custom_chunker.py
```

6.4 預期結果

預期輸出（chunk 數量與編號如實）

```
indexed 12 sentence-chunks
```

```
Q: What is a vector database?
```

```
[0.68] vector-databases#s0: # Vector Databases
```

```
A vector database stores embeddings — dense numeric vectors ...
```

```
[0.55] vector-databases#s1: Given a query vector, the database returns ...
```

註：同樣三份文件，固定大小切分產出 6 個 chunks，句子切分產出 12 個（每份 4 個），chunk id 變成 doc#s0 格式。粒度變細會影響檢索行為，這正是可以實驗的維度。

6.5 觀念重點

- Chunk 粒度是 RAG 最重要的超參數之一：太粗會混入雜訊，太細會切斷語意。
- 先讓預設流程跑通，再一次只換一個模組做對比——這是調 RAG 的正確姿勢。

七、範例五：05_shop_assistant.py —— 銷售網站的 RAG 實戰

7.1 這個範例要學什麼

把 RAG 放進一個具體的產品場景：虛擬 3C 賣場（data/shop/）。你會看到同一個 pipeline 如何支撐銷售網站上三個真實功能：客服問答、語意商品搜尋、政策問答。

7.2 賣場文件

data/shop/ 下有四份繁體中文文件，預設切分後產生 4 個 chunks：

```
products.md    商品目錄（8 項商品：筆電、手機、耳機、鍵盤、螢幕…）
shipping-returns.md 運送與退貨政策（運費、7 天鑑賞期、保固）
faq.md         常見問題（訂單查詢、分期、發票、取貨）
reviews.md     顧客評價摘錄
```

註：文件是中文。預設的 all-MiniLM-L6-v2

以英文為主，中文文件建議換多語言模型：RAG_EMBEDDING_MODEL=paraphrase-multilingual-MiniLM-L12

-v2。這正是「embedding 模型的訓練語料要跟文件語系匹配」的實例。

7.3 落點與實作

腳本用的還是同一個 RAGPipeline，只是把文件目錄換成 data/shop：

關鍵程式碼

```
settings = Settings()
settings.docs_dir = "data/shop" # 換成賣場文件
pipeline = RAGPipeline(settings)
```

三個落點的差別只在「問什麼問題」：

- 客服問答：「AirBook 14 適合帶出門工作嗎？」→ 檢索商品規格＋顧客評價；在網站上就是聊天機器人，答不出來轉真人客服。
- 語意商品搜尋：「預算一萬以內、降噪好的耳機」→ 不用關鍵字比對，直接對應到 SoundPods Pro；適合搜尋框或「幫我挑」功能。
- 政策問答：「買了不喜歡可以退嗎？幾天內？」→ 引用退貨政策回答；政策更新只要換文件重建索引，不用改程式。

7.4 執行

```
python examples/05_shop_assistant.py          # 預設示範三個場景
python examples/05_shop_assistant.py --ask "..." # 問自己的問題
python examples/05_shop_assistant.py --ask "..." --answer # 加上 LLM 生成答案（需 Ollama）
```

7.5 預期結果

預期輸出（分數為示意，chunk 內容如實）

已索引 4 個 chunks（賣場文件）

場景一：客服問答

網站這樣用：客服聊天機器人收到商品問題時，先檢索規格＋評價再回答…

Q: AirBook 14 適合帶出門工作嗎？

[0.72] Products (data/shop/products.md)

商品目錄

AirBook 14 輕薄筆電

- 價格：NT\$32,900

- 重量：1.2 公斤，14 吋螢幕 …

[0.61] Reviews (data/shop/reviews.md)

「實測續航約 16 小時，帶出門一整天不用充電…」 ★★★★★

7.6 觀念重點

- 同一個 pipeline，換文件就換場景：RAG 的能力邊界由「餵了什麼文件」決定。
- 客服場景的關鍵是引用來源：答案錯了要能追溯到哪份文件，政策更新才敢直接上線。
- 語意搜尋是 RAG 最被低估的用法：不需要 LLM，把 retrieval 單獨拿出來就是商品搜尋引擎。

八、常見問題與下一步

分數多少算好？

看相對排名，不看絕對值。0.7

在一個語料庫是第一名，在另一個可能排第五。調參時比較的是「同一個問題在不同設定下的排名變化」。

檢索品質不好，先調什麼？

順序：chunking（大小／重疊／策略）→ top-k → prompt 寫法 → 最後才換 embedding 模型。實務上八成的問題出在切分。

不想裝 torch / 沒有 GPU？

實作 Embedder 協定（embed(texts) -> np.ndarray），改呼叫 OpenAI embeddings API 即可，pipeline 其他部分不用動。

下一步可以玩什麼？

- Rerank：先用向量取回 20 個，再用 cross-encoder 精排取前 4 個。
- Hybrid search：BM25 關鍵字搜尋＋向量搜尋混排，專有名詞多的語料特別有效。
- 評測集：準備 10 個問答，看答對率——這正是 12 週課綱 Week 4 的驗收標準。
- 持久化：把 InMemoryVectorStore 換成 Chroma 或 pgvector，索引不用每次重建。